

Pattern Recognition Project '26

Movie Popularity Analysis

Milestone 1 & 2 Report

Team ID: CS_1

Delivery Date: 10/5/2026 – 8 AM

Instructor: Dr. Dina Khattab

TA: Eng. Hazem Yousef

1. Introduction

Milestone 1 focuses on building a complete machine learning pipeline to predict how popular a movie will be. We take raw movie data, clean it up, and train two different predictive models to see which one performs the best.

2. Data Preprocessing & Feature Engineering

In this phase, we transformed the raw dataset into a clean, numerical format to ensure data integrity and speed up the model's training process.

- **Data Cleaning & Integrity**

- **Action:** Removed duplicates and dropped rows with missing popularity.
- **Reason:** Duplicates introduce bias, and training on rows without a target value is impossible and misleading for the model.

- **Feature Extraction & Engineering**

- **Action:** Engineered numerical flags from metadata (**has_backdrop**, **has_homepage**, **has_poster**, **has_tagline**), and created a binary **is_title_changed** flag by comparing the title with the **original_title**. We also converted text into length metrics (**title_length**, **overview_length**) and booleans into integers (**0 and 1**).
- **Reason:** These features provide indirect signals about a movie's presentation and marketing. For example, the presence of posters or backdrops may indicate higher production or promotional effort, while text lengths (**overview_length**) may reflect content richness.

- **Temporal Feature Extraction**

- **Action:** Converted the **release_date** string into a datetime object, then extracted the **release_day**, **release_month**, and **release_year** into separate columns.
- **Reason:** Decomposing the date into discrete numerical components (year, month, day) translates time into a format the model can use to independently evaluate seasonal trends.

- **Handling Multi-label Categorical Data**

- **Action:** For complex string lists (**genres, production_companies, production_countries, spoken_languages**) we extracted the item count (numOf_) and primary item (First_).
- **Reason:** Multi-label strings (like "**Action, Comedy, Sci-Fi**") are highly unique and impossible to encode directly. Breaking them down tells the model if diversity (number of genres) or a *specific category* (the primary genre) affects the movie's popularity.

- **Feature Selection (Manual) / Dropping Useless Features**

- **Action:** Dropped IDs, raw text/URLs, and original multi-label columns.
- **Reason:** IDs are purely sequential and act as random noise. Removing raw text also avoids the need for complex NLP processing.

- **Handling Large Values (Skewness Correction)**

- **Action:** Applied **np.log1p** to highly skewed features (**revenue, budget, vote_count**).
- **Reason:** Normalizes extreme numerical ranges into a Gaussian-like distribution, crucial for linear models.

- **Handling Missing Values & Categorical Encoding**

- **Action:** Replacing missing values (**Mean** for **numerical**, **Mode** for **categorical**) and applying frequency encoding to the top **50** categories (grouping the rest as "**Other**").
- **Reason:** Mean/Mode imputation preserves valuable data rows. Frequency Encoding avoids high dimensionality caused by One-Hot Encoding, keeping the dataset dense and computationally efficient.

- **Outliers Treatment**

- **Action:** Applied clipping (Winsorization) to all numerical columns, capping values at the 1st percentile (**lower bound**) and 99th percentile (**upper bound**).
- **Reason:** Limits the impact of extreme outliers on the regression line and Mean Squared Error (MSE).

- **Feature Scaling (Standardization)**

- **Action:** Applied Z-score normalization (**x - mean**) / **std** to all numerical features.

- **Reason:** Ensures features operate on the same scale, preventing variables with massive magnitudes (like budget) from dominating the weights.

- **Feature Selection (Automated)**

-**Action:** Used **SelectKBest (f_regression)** to isolate the top **20** most significant features.

-**Reason:** Reduces noise, prevents overfitting, and significantly improves training efficiency, **f_regression** measures the linear relationship between each feature and the target variable.

- **Data Splitting**

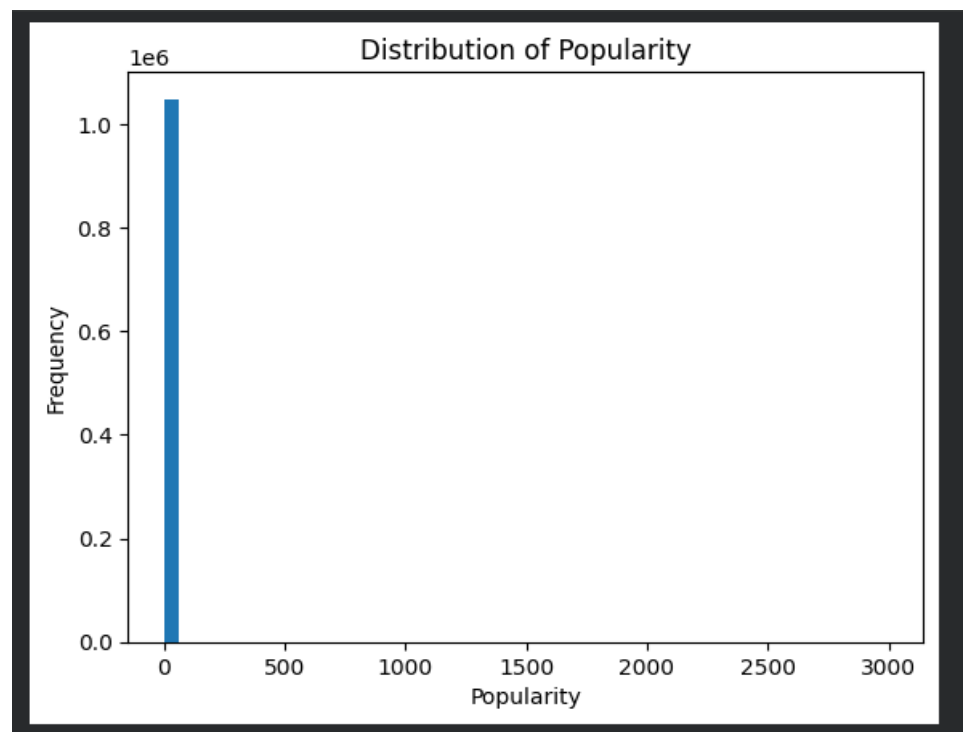
We used an 80/20 split for training and testing. Note that a separate validation set was not explicitly created, as cross-validation was used internally during model training.

- **Distribution of Popularity**

The distribution is highly **right skewed**.

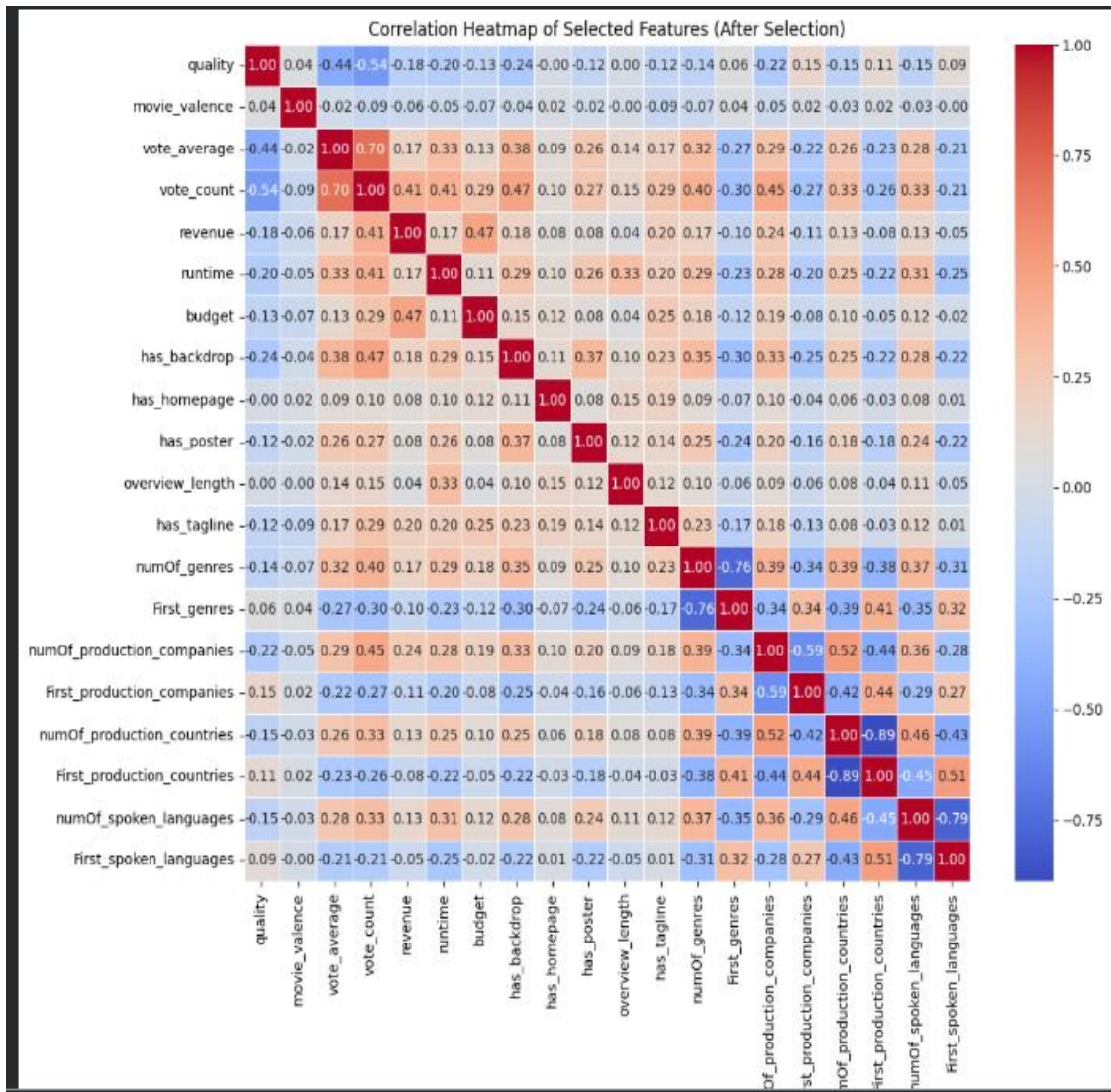
Most movies have **low popularity**.

Few extreme **outliers** exist.



Correlation Heatmap

The heatmap shows that **vote_count**, **revenue**, and **budget** have the strongest **positive correlation** with **popularity**. Most other features have **weak correlations**, indicating that popularity depends on multiple factors rather than a single feature.



3. Regression Models We Used

In This Phase, we apply two regression models (**Random Forest & Decision Tree**) were trained on the preprocessed data. Their performance was evaluated using **MAE**, **RMSE**, and **R²**, along with visualization techniques such as **Actual vs Predicted**, **Error Distribution**, **Residuals** and **Feature Importance** plots. The models were then compared to select the best performing one.

- **Random Forest**

An ensemble model that combines multiple decision trees to handle complex patterns and improve accuracy.

- **Decision Tree**

A model that splits the data into smaller subsets based on feature values, forming a tree-like structure. It captures non-linear relationships and is easy to interpret but may overfit if not controlled.

Models Evaluations

Model	MAE Train	RMSE Train	R ² Train
Random Forest	0.4586	2.8816	0.8299
Decision Tree	0.5107	5.5659	0.3655

Model	MAE Test	RMSE Test	R ² Test
Random Forest	0.5200	6.4667	0.4539
Decision Tree	0.5267	7.3917	0.2865

From the evaluations, we conclude that the **Random Forest** model performs better than the **Decision Tree**. It achieves lower error values (MAE and RMSE) and higher R² on both training and testing sets, indicating better accuracy and generalization. Therefore, **Random Forest** is selected as the best model for predicting movie popularity.

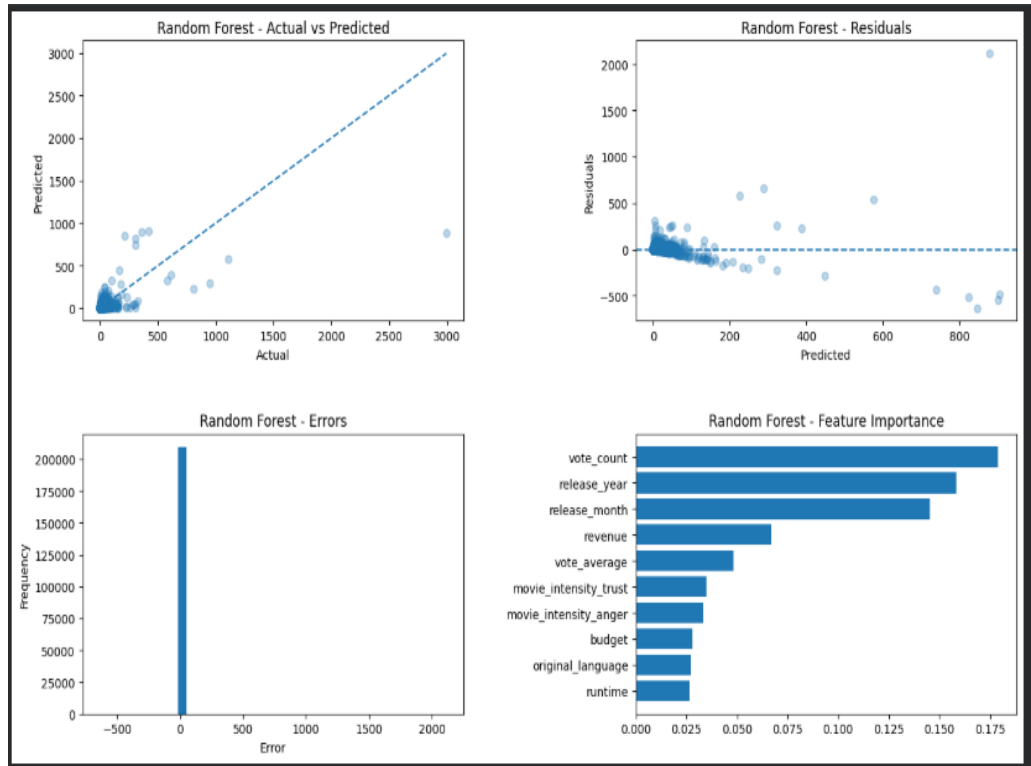
Models Visualizations

Random Forest Model

The model performs better with low popularity values but struggles with **high-value outliers**.

Error distribution is centered around zero but shows a long tail due to outliers.

Feature importance shows that variables like **release_year**, **vote_count**, and **revenue** have the strongest influence on predictions.

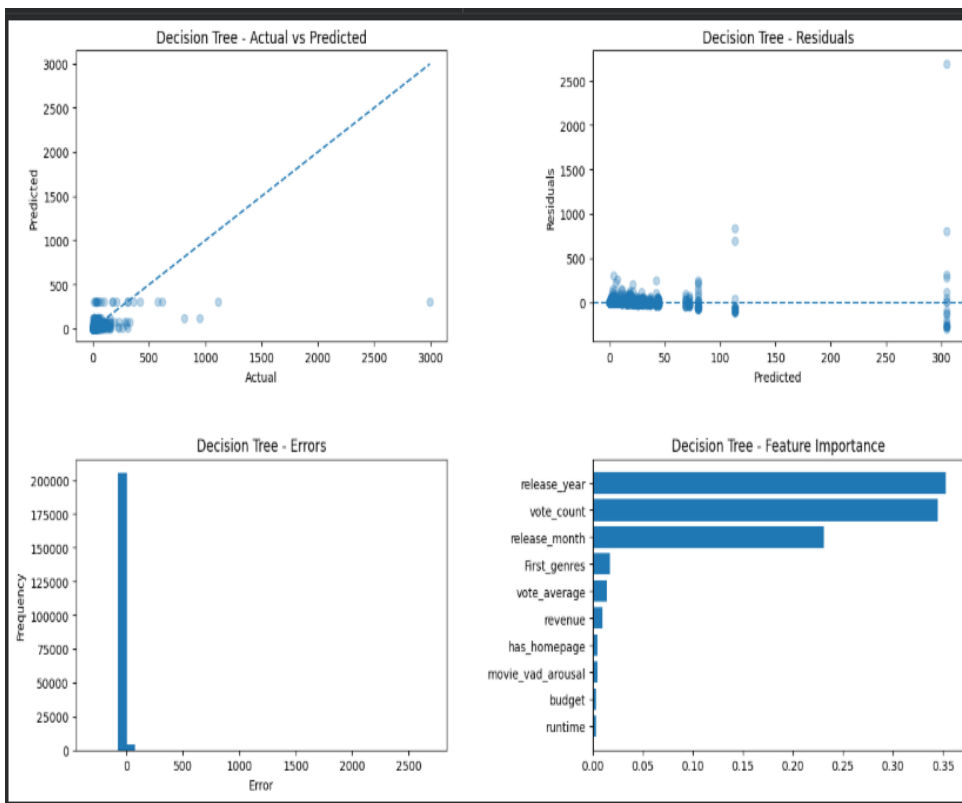


Decision Tree Model

The model performs well for low popularity values but is less accurate for **high outliers**.

Error distribution is centered around zero with more variation.

Feature importance shows **release_year** and **vote_count** as the most important features.



Conclusion & Intuition

Initial Intuition:

At the beginning of this project, our primary intuition was that a movie's **budget** was the most **critical factor** in determining its **popularity**. We assumed that **larger budgets** naturally translated to **better** production quality and wider promotional reach. We also expected that a **movie's "genres"** would have a simple and direct impact on its expected success.

Analysis Findings (Proved vs. Disproved):

Through our data analysis and model training, this initial intuition was partially disproved, revealing the **true complexity** of the problem. The Correlation Heatmap **demonstrated** that features like **vote_count** and **revenue** have a much **stronger positive correlation** with **popularity** than **budget**. Furthermore, the **Feature Importance** plots for both models confirmed that **release_year** and **vote_count** are the actual primary drivers for predictions. This proved to us that a **movie's popularity** is not merely a financial formula based on how much money was spent, but a complex statistical phenomenon heavily influenced by actual audience engagement and temporal factors.

Conclusion:

In conclusion, this phase of the project successfully established a robust data preprocessing pipeline. By effectively handling **missing values**, **correcting skewness**, **treating extreme outliers**, and utilizing automated feature selection to isolate the top **20 features**.

----- Milestone 2 -----

Introduction

In phase 2, we are shifting from **Regression** to **Multiclass Classification** to categorize **movies** into **four** clear tiers of success: **Very Low, Low, Medium, and High**.

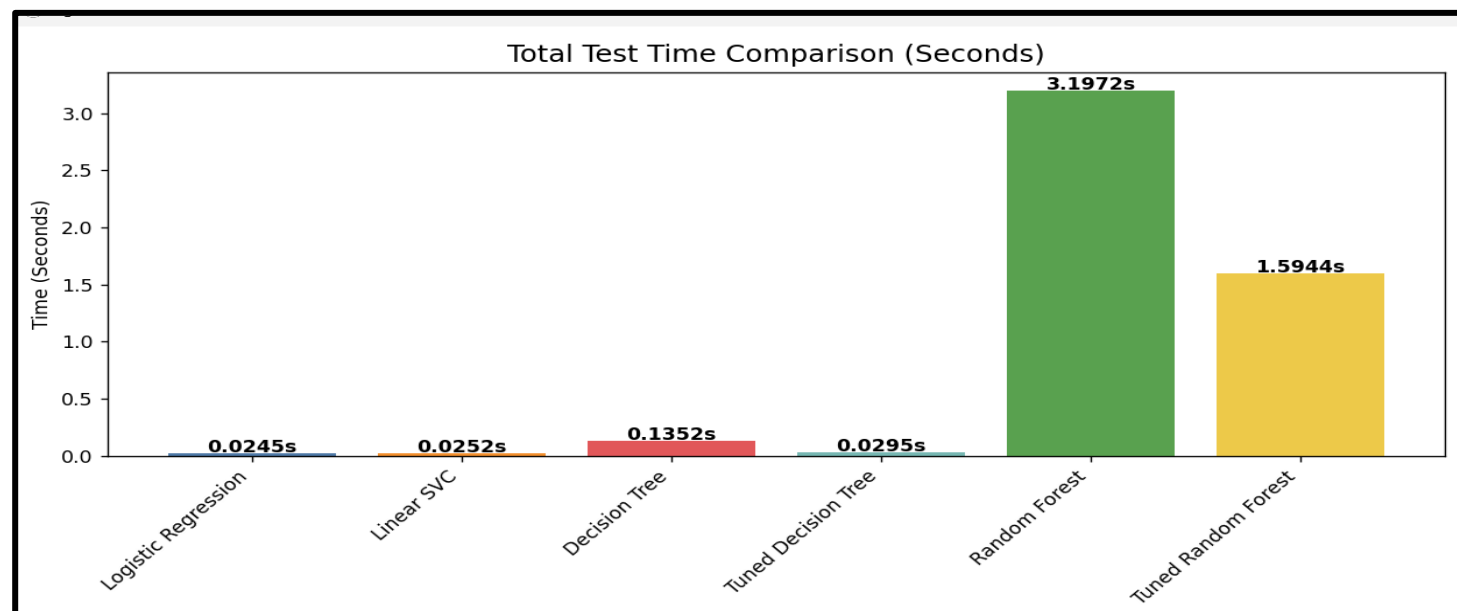
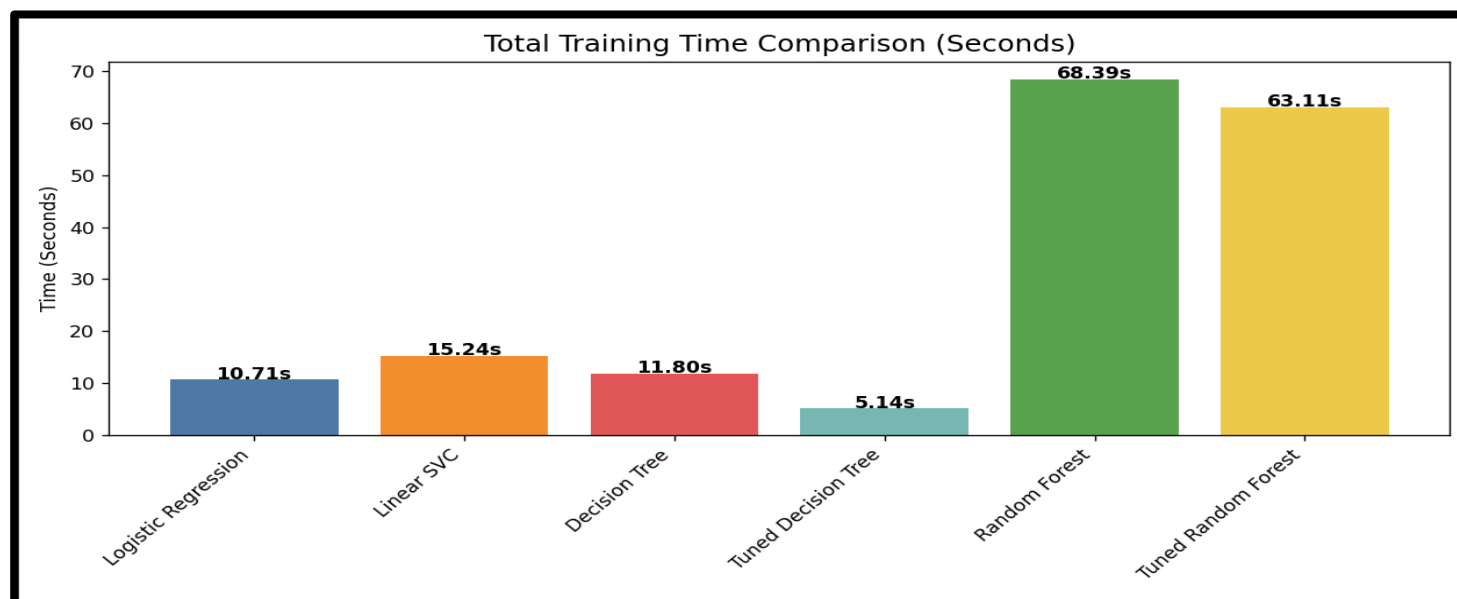
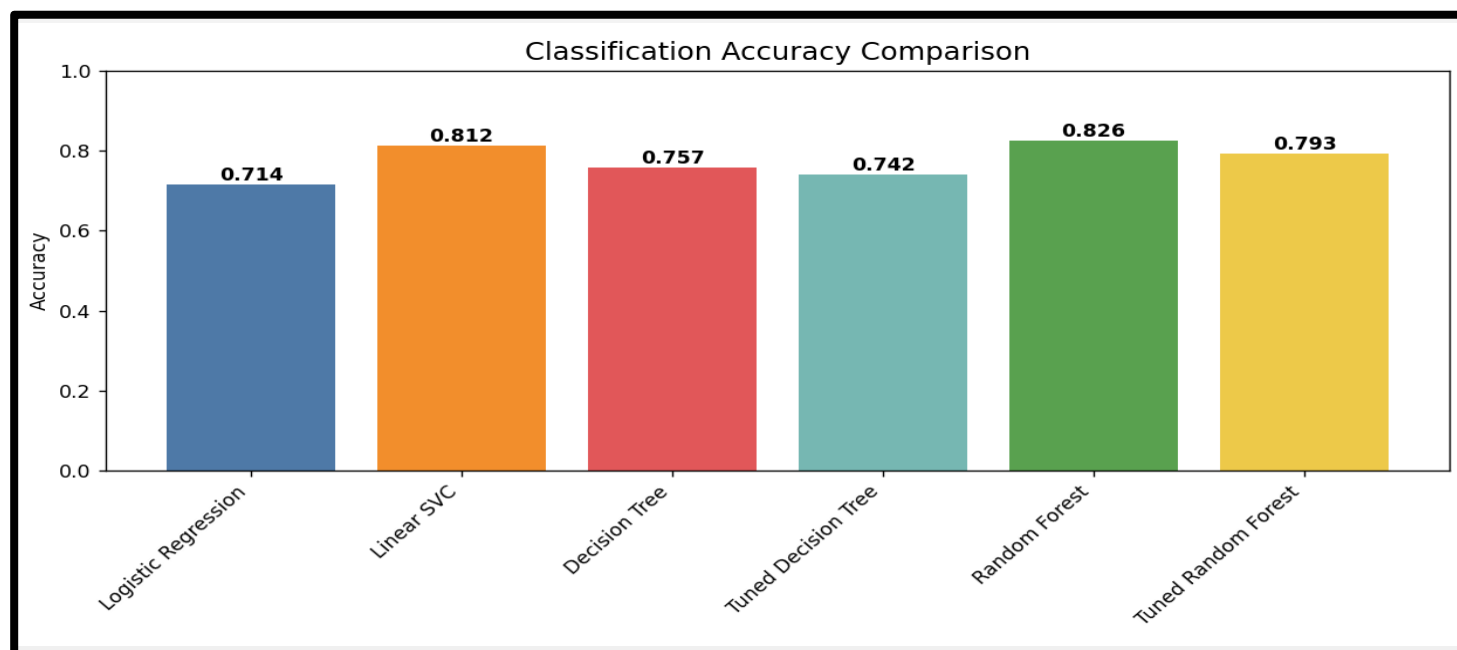
Feature Selection (Classification vs. Regression)

In the **Regression phase (Milestone 1)**, we retained most features, because Decision Tree and Random Forest models handle high dimensionality well for continuous targets. However, for this **Classification phase**, the target is categorical (**popularityLevel**). Therefore, our selection process had to adapt:

- **Noise Reduction:** We first identified and dropped **constant features** (zero variance) that provide no discriminative power.
- **Statistical Filtering:** We applied SelectKBest using the (**f_classif**) scoring function, which is mathematically suited for categorical targets (unlike **f_regression**), selecting the top **20** most informative features
- **Proved or Disproved:** By reducing the feature space to the top **20**, we eliminated noise, significantly sped up the training time for models like Linear SVC and Logistic Regression, and enabled our **Random Forest** to achieve an accuracy of **82.59%** without suffering from high dimensionality.

Classification Results & Visualizations

We trained and evaluated **6 classification models** to categorize **movie popularity**. The performance of these models is summarized based on their **Accuracy, Total Training Time, and Total Test Time**.

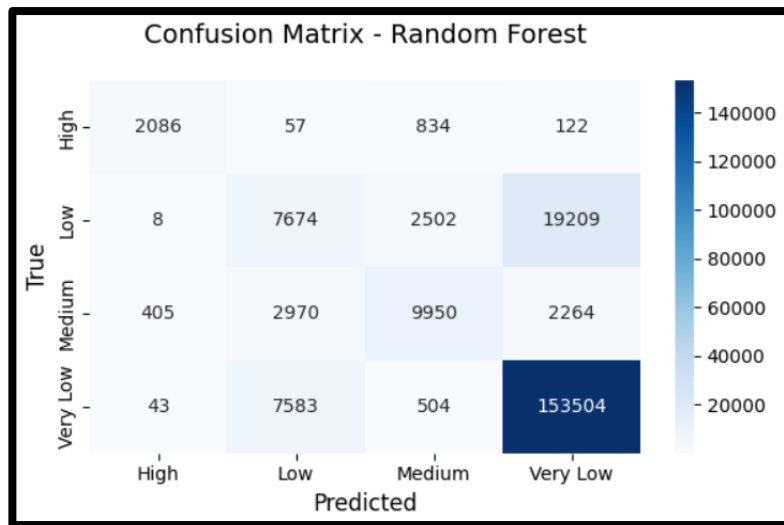
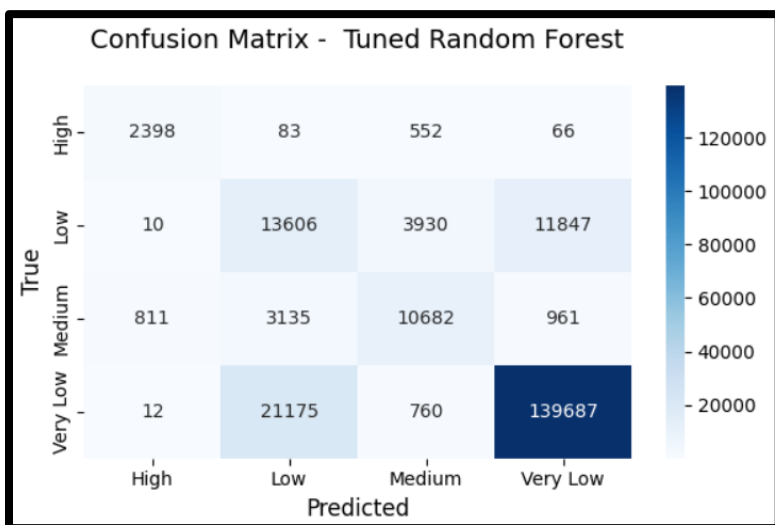


Conclusion From Graphs:

- **Highest Accuracy:** The **Base Random Forest** outperformed all models, achieving an accuracy of **82.59%**.
- **Training Efficiency:** The Tuned Decision Tree was the fastest to train (5.14s), making it highly efficient.
- **Testing Speed:** Linear models (**Logistic Regression&Linear SVC**) and the Tuned Decision Tree were the fastest during testing (approx. 0.03 seconds), ideal for real-time predictions.

Confusion Matrices Analysis

To look beyond raw accuracy, we analyzed the **Confusion Matrices** for our models, particularly focusing on our best performing algorithms (Base Random Forest vs. Tuned Random Forest).



Conclusion From Matrices:

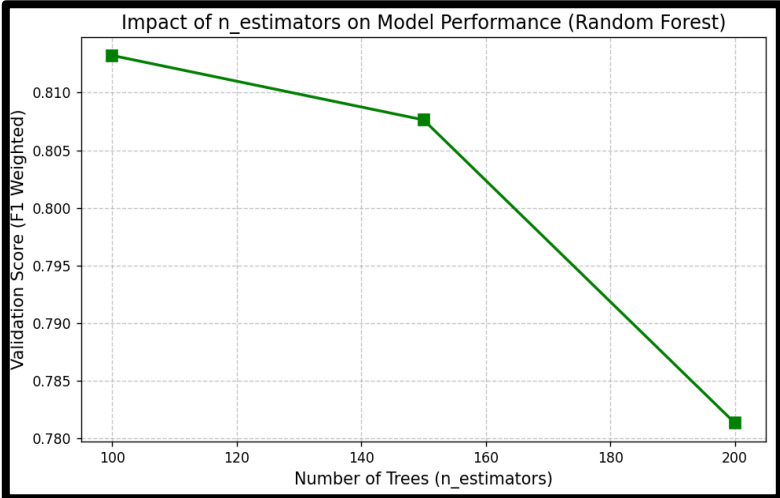
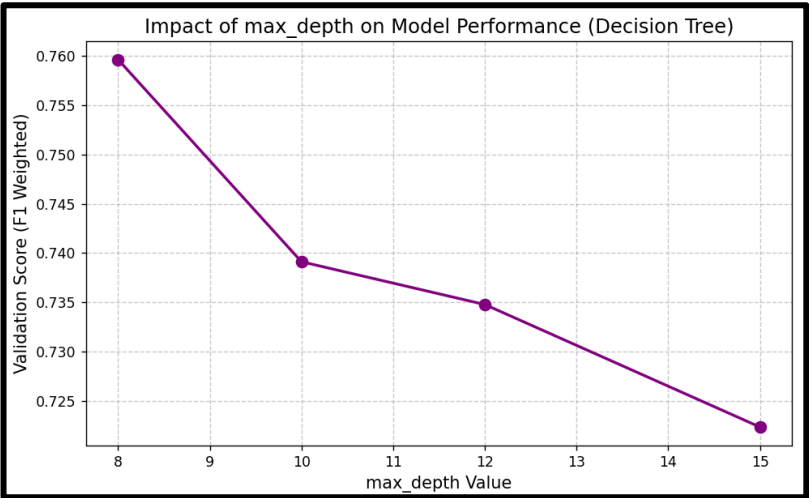
- **Base Model (Overfitting):** Although both models used `class_weight='balanced'`, the **Base Random Forest** overfits the training data due to **unlimited tree depth**, inflating overall accuracy by favoring the majority class.
- **Tuned Model (Generalization):** By restricting tree complexity (`max_depth=20`) and optimizing `f1_weighted`, the Tuned model successfully generalized. It sacrificed a bit of raw accuracy but drastically improved minority class detection (correct '**Low**' predictions jumped from **7,674 to 13,606**).

Hyperparameter Tuning & Cross-Validation Impact

- To ensure our models generalize well and do not simply memorize the training data, we applied **hyperparameter tuning** coupled with **Cross-Validation**. We utilized **GridSearchCV** (with **cv=3**) for the **Decision Tree** and **RandomizedSearchCV** (with **cv=2**) for the **Random Forest**, specifically optimizing for the **f1_weighted** metric.
- The Role of Cross-Validation:**

Model	Hyperparameter	Hyperparameter Value	Validation Score
Decision Tree	max_depth	8	0.7596
Decision Tree	max_depth	10	0.7391
Decision Tree	max_depth	12	0.7347
Decision Tree	max_depth	15	0.7223
Random Forest	n_estimators	100	0.8132
Random Forest	n_estimators	150	0.8076
Random Forest	n_estimators	200	0.7813

By using **cross-validation** during the tuning process, we ensured that our selected **hyperparameters** were robust. This **technique** validates the **model's performance** across multiple folds of the training data, guaranteeing that our results are not just a lucky outcome of a single train-test split.



Conclusion from Graphs:

Decision Tree: Increasing the depth beyond **8** clearly leads to overfitting, as the validation score steadily drops from **~0.76** to **~0.72**.

Random Forest: Adding more trees does not always guarantee better results. The model peaked at **100** trees (**~0.81**). Increasing it to **200** trees introduced noise and caused the score to drop to **~0.78**.

Conclusion & Project Intuition

- **Initial Intuition:**

At the beginning of this project, we initially assumed that a **movie's financial** backing (**budget**) and its earnings (**revenue**) would be the sole dictators of its **popularity level**. We also intuitively suspected that without proper constraints, **complex models** might struggle with the imbalanced nature of our dataset.

- **Analysis Findings (Proved vs. Disproved):**

Disproved: The assumption that financial metrics are the only key drivers was completely **disproved**. Our models revealed that categorical metadata (such as **genres**, **production companies**, and **release dates**) carry massive **predictive** weight. **Popularity** is clearly driven by a combination of content and timing, not just financial numbers.

Proved: Our suspicion regarding model behavior was **proved** true. The base models with unrestricted complexity (like the **base Random Forest**) overfit the data by memorizing the majority class. However, implementing **hyperparameter tuning** and **cross-validation** successfully cured this, proving that constraining the models is essential for real-world generalization.

- **Conclusion:**

Milestone 2 successfully transformed our approach from a **continuous prediction** to a **Multiclass Classification** task. By addressing class **imbalance** and optimizing **hyperparameters**, the **Tuned Random Forest** proved to be the most robust choice for handling the **nonlinear** metadata of movies. Ultimately, this phase demonstrated that predicting movie popularity requires a **well-regularized** ensemble model that looks beyond just the **budget** and **revenue**.